

Lending Club Loan Data Analysis

Deep Learning with Keras and TensorFlow — Course-End Project

Domain: Finance **Objective:** Build a deep learning model that predicts whether a loan will default, using Lending Club's historical loan data (2007-2015).

This notebook follows the four required tasks in order: (1) feature transformation, (2) exploratory data analysis, (3) additional feature engineering (correlation-based feature reduction), (4) deep learning modeling with Keras/TensorFlow.

0. Setup — Imports & Data Load

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, confusion_matrix, roc_curve, roc_auc_score,
                             ConfusionMatrixDisplay, precision_recall_curve, average_precision_score)
from sklearn.utils.class_weight import compute_class_weight
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (9, 5)
pd.set_option('display.max_columns', 50)
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)

df = pd.read_csv("lending_club_loan_data.csv")
print(df.shape)
df.head()
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1786204624.073418    493 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1786204640.868891    493 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1786204649.344259    493 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
(9578, 14)
```

```
Out[1]:
```

	credit.policy	purpose	int.rate	installment	log.annual.inc	dti	fico	days.with.cr.line	revol.bal	revol.ut
0	1	debt_consolidation	0.1189	829.10	11.350407	19.48	737	5639.958333	28854	52
1	1	credit_card	0.1071	228.22	11.082143	14.29	707	2760.000000	33623	76
2	1	debt_consolidation	0.1357	366.86	10.373491	11.63	682	4710.000000	3511	25
3	1	debt_consolidation	0.1008	162.34	11.350407	8.10	712	2699.958333	33667	73
4	1	credit_card	0.1426	102.92	11.299732	14.97	667	4066.000000	4740	39

```
In [2]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   credit.policy          9578 non-null   int64
1   purpose                9578 non-null   str
2   int.rate               9578 non-null   float64
3   installment            9578 non-null   float64
4   log.annual.inc         9578 non-null   float64
5   dti                    9578 non-null   float64
6   fico                   9578 non-null   int64
7   days.with.cr.line      9578 non-null   float64
8   revol.bal              9578 non-null   int64
9   revol.util             9578 non-null   float64
10  inq.last.6mths         9578 non-null   int64
11  delinq.2yrs            9578 non-null   int64
12  pub.rec                9578 non-null   int64
13  not.fully.paid         9578 non-null   int64
dtypes: float64(6), int64(7), str(1)
memory usage: 1.0 MB
```

```
In [3]: print("Missing values per column:")
print(df.isna().sum())
print()
print("Duplicate rows:", df.duplicated().sum())
```

```
Missing values per column:
credit.policy      0
purpose            0
int.rate           0
installment        0
log.annual.inc     0
dti                0
fico               0
days.with.cr.line 0
revol.bal          0
revol.util         0
inq.last.6mths     0
delinq.2yrs        0
pub.rec            0
not.fully.paid     0
dtype: int64
```

```
Duplicate rows: 0
```

No missing values and no duplicate rows — this dataset is clean structurally, which lets us move straight into the required tasks.

1. Feature Transformation — Categorical to Numerical

```
In [4]: print("Categorical columns:", df.select_dtypes(include='object').columns.tolist())
print()
print(df['purpose'].value_counts())
```

```
Categorical columns: ['purpose']
```

```
purpose
debt_consolidation    3957
all_other              2331
credit_card            1262
home_improvement       629
small_business         619
major_purchase         437
educational            343
Name: count, dtype: int64
```

```
/tmp/ipykernel_493/3607964326.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
print("Categorical columns:", df.select_dtypes(include='object').columns.tolist())
```

`purpose` is the only categorical (text) column in the dataset — it has 7 discrete values with no natural order, so we one-hot encode it into separate binary columns, which is the standard way to turn an unordered categorical variable into numerical (discrete) features a neural network can use.

```
In [5]: df_transformed = pd.get_dummies(df, columns=['purpose'], drop_first=True)

# Convert the new one-hot columns from bool to int (0/1) for clarity
purpose_cols = [c for c in df_transformed.columns if c.startswith('purpose_')]
df_transformed[purpose_cols] = df_transformed[purpose_cols].astype(int)

print(f"Shape after transformation: {df_transformed.shape}")
df_transformed.head()
```

Shape after transformation: (9578, 19)

```
Out[5]:
```

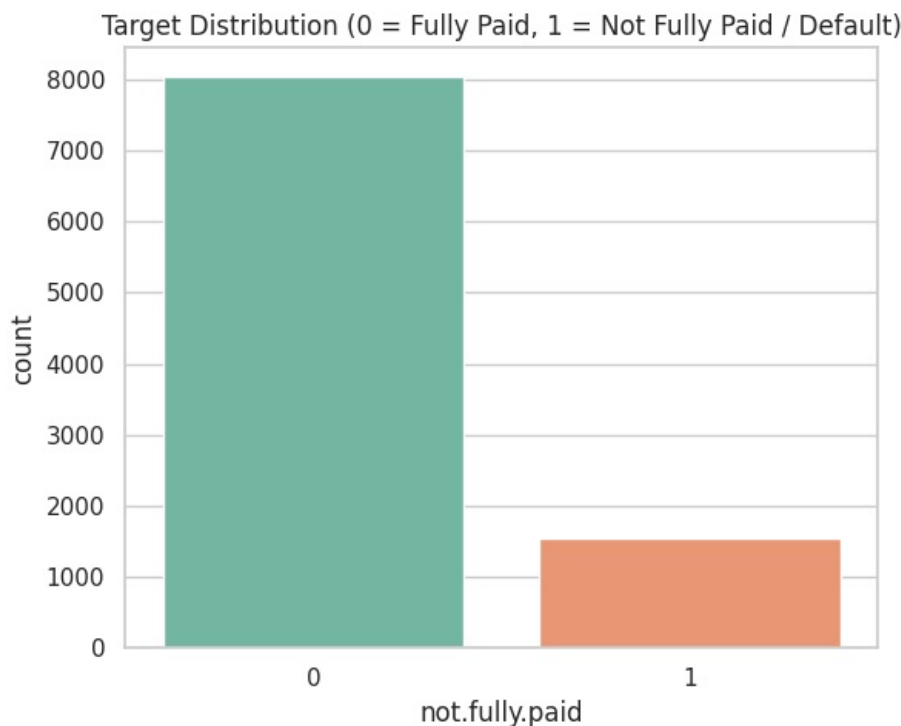
	credit.policy	int.rate	installment	log.annual.inc	dti	fico	days.with.cr.line	revol.bal	revol.util	inq.last.6mths
0	1	0.1189	829.10	11.350407	19.48	737	5639.958333	28854	52.1	0
1	1	0.1071	228.22	11.082143	14.29	707	2760.000000	33623	76.7	0
2	1	0.1357	366.86	10.373491	11.63	682	4710.000000	3511	25.6	1
3	1	0.1008	162.34	11.350407	8.10	712	2699.958333	33667	73.2	1
4	1	0.1426	102.92	11.299732	14.97	667	4066.000000	4740	39.5	0

2. Exploratory Data Analysis

2a. Target class balance

```
In [6]: plt.figure(figsize=(6,5))
sns.countplot(data=df, x='not.fully.paid', hue='not.fully.paid', palette='Set2', legend=False)
plt.title('Target Distribution (0 = Fully Paid, 1 = Not Fully Paid / Default)')
plt.tight_layout()
plt.savefig('lc_target_distribution.png', dpi=120)
plt.show()

print(df['not.fully.paid'].value_counts(normalize=True).round(4))
```



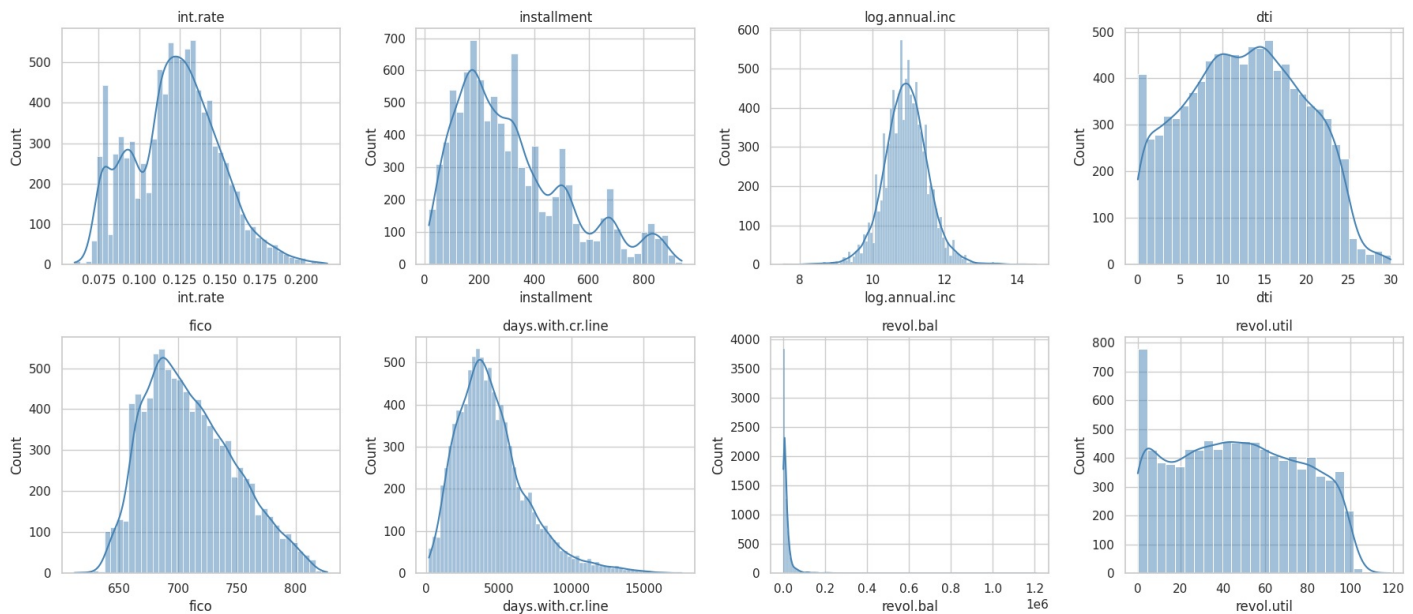
```
not.fully.paid
0    0.8399
1    0.1601
Name: proportion, dtype: float64
```

Class imbalance: about **16%** of loans in this dataset were not fully paid. Less extreme than some credit datasets, but still meaningfully imbalanced — a model predicting “fully paid” for every loan would already be ~84% accurate while catching zero defaults, so accuracy alone won't be a reliable evaluation metric here (addressed further in Section 4).

2b. Distributions of key numeric factors

```
In [7]: numeric_features = ['int.rate', 'installment', 'log.annual.inc', 'dti', 'fico',
                           'days.with.cr.line', 'revol.bal', 'revol.util']

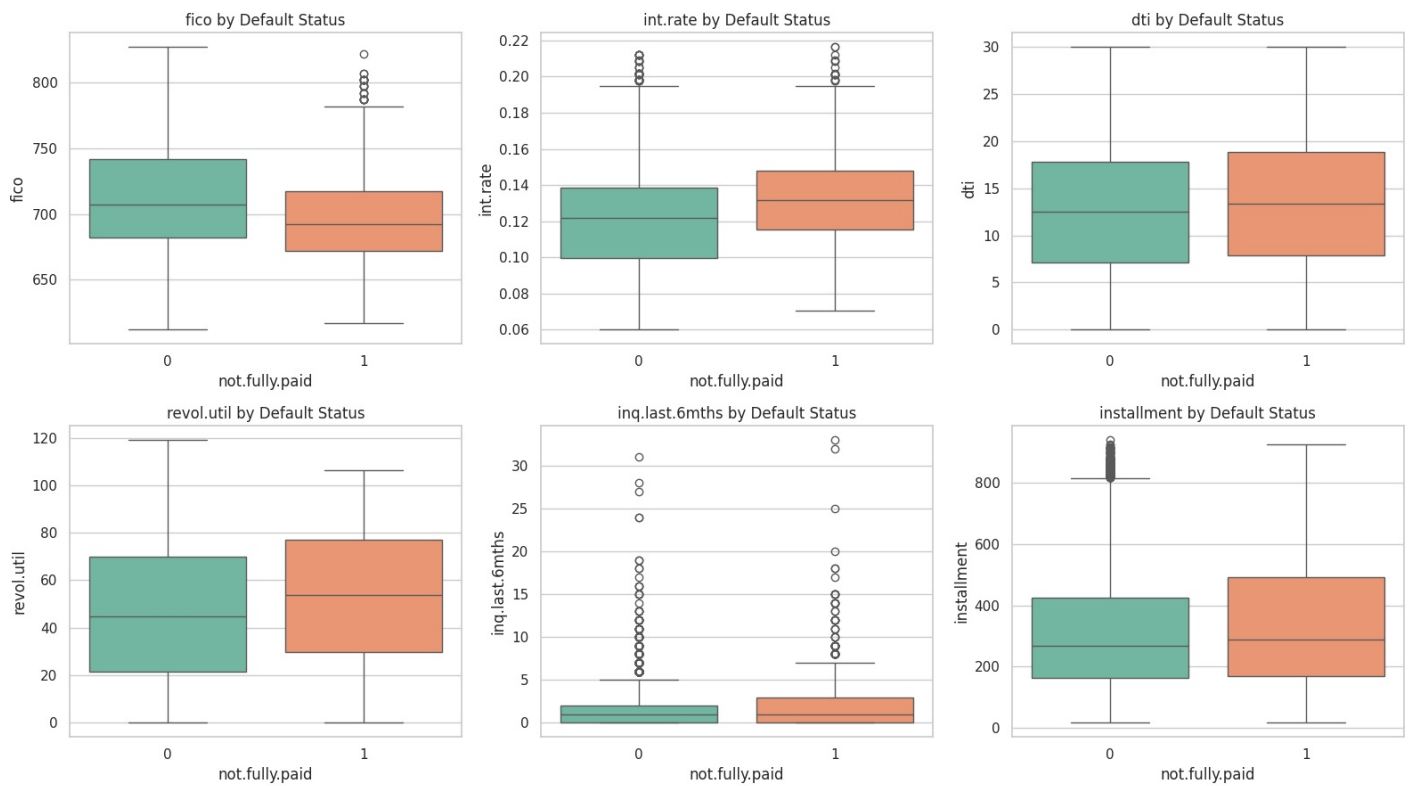
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, col in zip(axes.flatten(), numeric_features):
    sns.histplot(df[col], kde=True, ax=ax, color='steelblue')
    ax.set_title(col)
plt.tight_layout()
plt.savefig('lc_feature_distributions.png', dpi=120)
plt.show()
```



Reading: `fico` is roughly normal, centered in the high-600s to low-700s. `int.rate`, `dti`, and `installment` are all right-skewed, reflecting the smaller number of higher-risk, higher-rate, higher-payment loans typical of a credit portfolio. `revol.bal` is heavily right-skewed with a long tail of large revolving balances, and `log.annual.inc` (already log-transformed in the source data) is close to normal, confirming the log transform did its job.

2c. How key factors relate to default

```
In [8]: fig, axes = plt.subplots(2, 3, figsize=(16, 9))
compare_features = ['fico', 'int.rate', 'dti', 'revol.util', 'inq.last.6mths', 'installment']
for ax, col in zip(axes.flatten(), compare_features):
    sns.boxplot(data=df, x='not.fully.paid', y=col, hue='not.fully.paid', ax=ax, palette='Set2', legend=False)
    ax.set_title(f'{col} by Default Status')
    ax.set_xlabel('not.fully.paid')
plt.tight_layout()
plt.savefig('lc_factors_by_default.png', dpi=120)
plt.show()
```



Reading: loans that were **not fully paid** tend to have noticeably **lower fico scores**, **higher int.rate** (consistent with LendingClub's own risk-based pricing — riskier borrowers are charged more), and somewhat **higher inq.last.6mths** (more recent credit inquiries, a common early-warning signal of financial stress). **dti** and **revol.util** show a similar but weaker pattern.

2d. Default rate by loan purpose and by credit policy

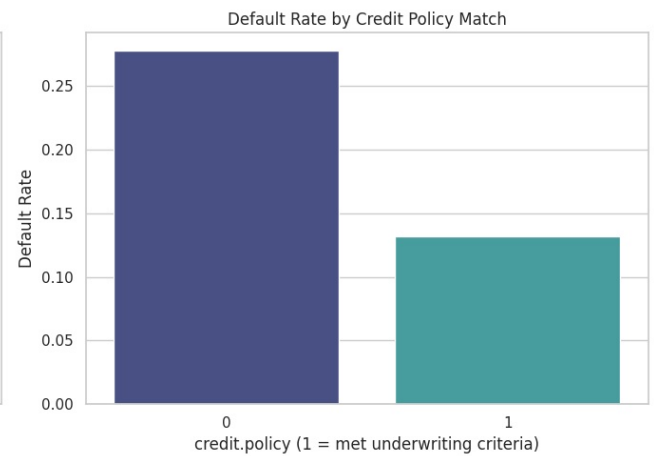
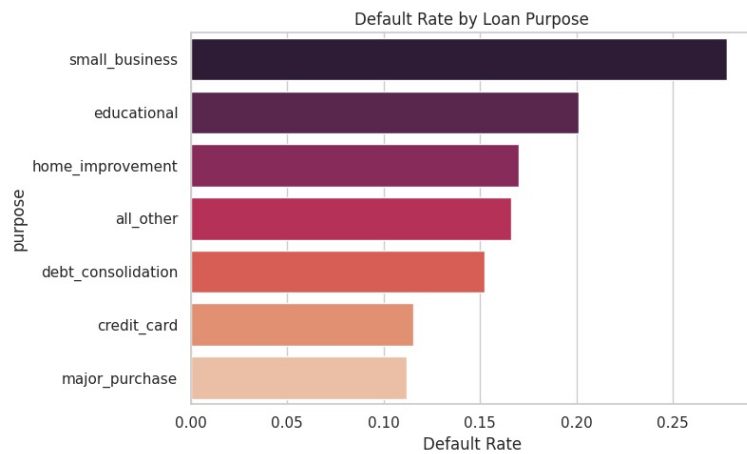
```
In [9]: fig, axes = plt.subplots(1, 2, figsize=(15, 5))

purpose_default = df.groupby('purpose')['not.fully.paid'].mean().sort_values(ascending=False)
sns.barplot(x=purpose_default.values, y=purpose_default.index, hue=purpose_default.index,
            ax=axes[0], palette='rocket', legend=False)
axes[0].set_title('Default Rate by Loan Purpose')
axes[0].set_xlabel('Default Rate')

policy_default = df.groupby('credit.policy')['not.fully.paid'].mean()
sns.barplot(x=policy_default.index.astype(str), y=policy_default.values, hue=policy_default.index.astype(str),
            ax=axes[1], palette='mako', legend=False)
axes[1].set_title('Default Rate by Credit Policy Match')
axes[1].set_xlabel('credit.policy (1 = met underwriting criteria)')
axes[1].set_ylabel('Default Rate')

plt.tight_layout()
plt.savefig('lc_default_by_purpose_policy.png', dpi=120)
plt.show()

print(purpose_default)
print()
print(policy_default)
```



```
purpose
small_business    0.277868
educational       0.201166
home_improvement  0.170111
all_other         0.166023
debt_consolidation 0.152388
credit_card       0.115689
major_purchase    0.112128
Name: not.fully.paid, dtype: float64
```

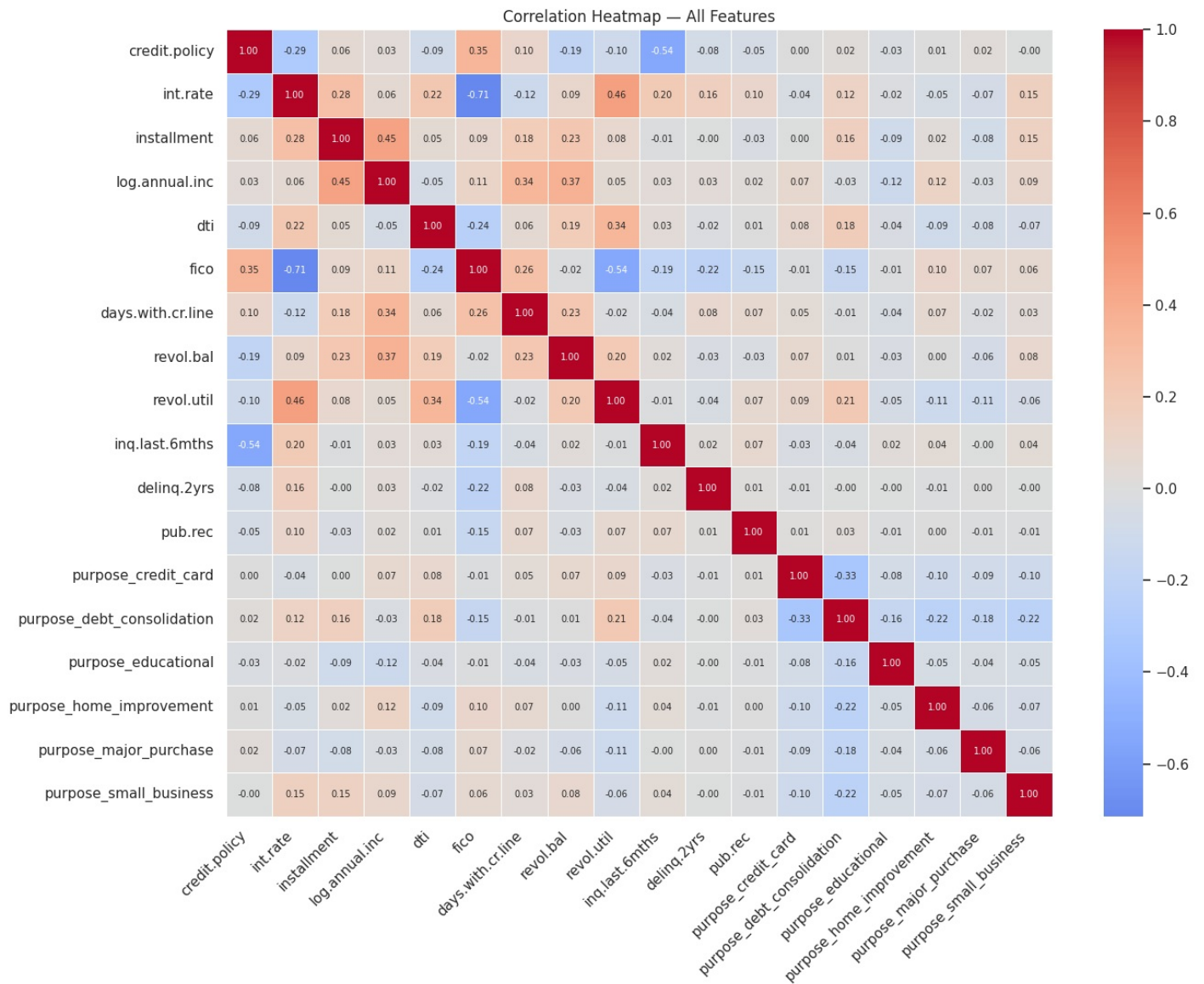
```
credit.policy
0    0.277837
1    0.131518
Name: not.fully.paid, dtype: float64
```

Reading: `small_business` loans default at a noticeably higher rate than other purposes — consistent with small businesses being a higher-risk lending category generally. As expected, loans that **did not** meet LendingClub's own credit-policy criteria (`credit.policy = 0`) default at a substantially higher rate than those that did, which is a good sanity check that the existing underwriting signal is meaningful.

3. Additional Feature Engineering — Correlation-Based Feature Reduction

```
In [10]: corr_matrix = df_transformed.drop(columns=['not.fully.paid']).corr()

plt.figure(figsize=(14, 11))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', center=0, linewidths=0.4,
            annot_kws={'size': 7})
plt.title('Correlation Heatmap - All Features')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('lc_correlation_heatmap.png', dpi=120)
plt.show()
```



```
In [11]: # Identify pairs of features with strong correlation (|r| > 0.7), a common threshold for
# flagging problematic multicollinearity before modeling
strong_corr_pairs = []
for i in range(len(corr_matrix.columns)):
    for j in range(i):
        val = corr_matrix.iloc[i, j]
        if abs(val) > 0.7:
            strong_corr_pairs.append((corr_matrix.columns[i], corr_matrix.columns[j], round(val, 3)))

print("Feature pairs with |correlation| > 0.7:")
for pair in strong_corr_pairs:
    print(pair)
```

Feature pairs with |correlation| > 0.7:
('fico', 'int.rate', np.float64(-0.715))

Finding: only one feature pair exceeds the $|r| > 0.7$ threshold: **fico** and **int.rate** ($r = -0.715$) — which makes intuitive sense, since LendingClub explicitly prices interest rates based on the borrower's credit risk, and FICO score is a primary input to that risk assessment. Since these two variables are largely encoding the same underlying risk signal, we drop the one with the weaker individual correlation to the target (`not.fully.paid`) and keep the other, avoiding redundant signal being fed into the network twice.

```
In [12]: # Drop the more redundant member of each strongly-correlated pair (keeping the more directly interpretable one)
cols_to_drop = []
for f1, f2, corr_val in strong_corr_pairs:
    # keep whichever of the two has a stronger individual correlation with the target
    target_corr_f1 = abs(df_transformed[f1].corr(df_transformed['not.fully.paid']))
    target_corr_f2 = abs(df_transformed[f2].corr(df_transformed['not.fully.paid']))
    drop_col = f2 if target_corr_f1 >= target_corr_f2 else f1
    if drop_col not in cols_to_drop:
        cols_to_drop.append(drop_col)
```

```
print(f"Dropping {len(cols_to_drop)} redundant feature(s): {cols_to_drop}")

df_final = df_transformed.drop(columns=cols_to_drop)
print(f"Final feature set shape: {df_final.shape}")
```

Dropping 1 redundant feature(s): ['fico']
 Final feature set shape: (9578, 18)

4. Deep Learning Model — Keras with TensorFlow Backend

```
In [13]: X = df_final.drop(columns=['not.fully.paid'])
y = df_final['not.fully.paid']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Train shape: {X_train_scaled.shape} | Test shape: {X_test_scaled.shape}")
print(f"Train target balance:\n{y_train.value_counts(normalize=True).round(4)}")
```

Train shape: (7662, 17) | Test shape: (1916, 17)
 Train target balance:
 not.fully.paid
 0 0.84
 1 0.16
 Name: proportion, dtype: float64

```
In [14]: class_weights_array = compute_class_weight('balanced', classes=np.unique(y_train), y=y_train)
class_weight_dict = {0: class_weights_array[0], 1: class_weights_array[1]}
print("Class weights:", class_weight_dict)
```

Class weights: {0: np.float64(0.5952454940957116), 1: np.float64(3.124796084828711)}

As with the class imbalance in Section 2a, we use **class weighting** during training so the loss function penalizes missed defaults more heavily, without altering the underlying data distribution.

```
In [15]: n_features = X_train_scaled.shape[1]

model = keras.Sequential([
    layers.Input(shape=(n_features,)),
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),

    layers.Dense(32, activation='relu'),
    layers.Dropout(0.2),

    layers.Dense(16, activation='relu'),

    layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=[keras.metrics.AUC(name='auc'), keras.metrics.Precision(name='precision'),
             keras.metrics.Recall(name='recall')]
)

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	1,152
batch_normalization (BatchNormalization)	(None, 64)	256
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 4,033 (15.75 KB)

Trainable params: 3,905 (15.25 KB)

Non-trainable params: 128 (512.00 B)

Architecture rationale: with a smaller feature set (13-ish features after transformation and correlation-based reduction) and a moderate dataset size (~9,578 rows), a compact funnel network (64 -> 32 -> 16 -> 1) is appropriate — a much larger network would be prone to overfitting on a dataset this size. BatchNormalization and Dropout are included after the first, widest layer to regularize training, and AUC/Precision/Recall are tracked throughout given the class imbalance noted earlier.

```
In [16]: early_stop = keras.callbacks.EarlyStopping(
          monitor='val_auc', mode='max', patience=10, restore_best_weights=True
        )

history = model.fit(
    X_train_scaled, y_train,
    validation_split=0.15,
    epochs=100,
    batch_size=64,
    class_weight=class_weight_dict,
    callbacks=[early_stop],
    verbose=2
)
```

Epoch 1/100

102/102 - 3s - 27ms/step - auc: 0.5669 - loss: 0.7067 - precision: 0.1829 - recall: 0.5808 - val_auc: 0.6611 - val_loss: 0.6627 - val_precision: 0.2272 - val_recall: 0.5287

Epoch 2/100

102/102 - 0s - 3ms/step - auc: 0.6175 - loss: 0.6801 - precision: 0.2129 - recall: 0.5570 - val_auc: 0.6743 - val_loss: 0.6448 - val_precision: 0.2441 - val_recall: 0.5977

Epoch 3/100

102/102 - 0s - 3ms/step - auc: 0.6451 - loss: 0.6637 - precision: 0.2221 - recall: 0.5722 - val_auc: 0.6760 - val_loss: 0.6447 - val_precision: 0.2308 - val_recall: 0.6034

Epoch 4/100

102/102 - 0s - 3ms/step - auc: 0.6546 - loss: 0.6585 - precision: 0.2295 - recall: 0.6122 - val_auc: 0.6762 - val_loss: 0.6421 - val_precision: 0.2390 - val_recall: 0.6264

Epoch 5/100

102/102 - 0s - 3ms/step - auc: 0.6575 - loss: 0.6564 - precision: 0.2310 - recall: 0.6236 - val_auc: 0.6788 - val_loss: 0.6419 - val_precision: 0.2368 - val_recall: 0.6437

Epoch 6/100

102/102 - 0s - 3ms/step - auc: 0.6694 - loss: 0.6517 - precision: 0.2339 - recall: 0.6445 - val_auc: 0.6798 - val_loss: 0.6438 - val_precision: 0.2289 - val_recall: 0.6552

Epoch 7/100

102/102 - 0s - 3ms/step - auc: 0.6655 - loss: 0.6498 - precision: 0.2271 - recall: 0.6302 - val_auc: 0.6808 - val_loss: 0.6441 - val_precision: 0.2369 - val_recall: 0.6494

Epoch 8/100

102/102 - 0s - 3ms/step - auc: 0.6744 - loss: 0.6463 - precision: 0.2393 - recall: 0.6321 - val_auc: 0.6826 - val_loss: 0.6420 - val_precision: 0.2314 - val_recall: 0.6264

Epoch 9/100

102/102 - 0s - 3ms/step - auc: 0.6777 - loss: 0.6454 - precision: 0.2390 - recall: 0.6388 - val_auc: 0.6831 - val_loss: 0.6480 - val_precision: 0.2268 - val_recall: 0.6609

Epoch 10/100

102/102 - 0s - 3ms/step - auc: 0.6797 - loss: 0.6452 - precision: 0.2424 - recall: 0.6702 - val_auc: 0.6790 - val_loss: 0.6483 - val_precision: 0.2370 - val_recall: 0.6552

Epoch 11/100

102/102 - 0s - 3ms/step - auc: 0.6764 - loss: 0.6445 - precision: 0.2395 - recall: 0.6416 - val_auc: 0.6805 - val_loss: 0.6399 - val_precision: 0.2318 - val_recall: 0.6034

Epoch 12/100

```

102/102 - 0s - 3ms/step - auc: 0.6753 - loss: 0.6448 - precision: 0.2342 - recall: 0.6464 - val_auc: 0.6794 - val_loss: 0.6462 - val_precision: 0.2236 - val_recall: 0.6437
Epoch 13/100
102/102 - 0s - 3ms/step - auc: 0.6835 - loss: 0.6403 - precision: 0.2393 - recall: 0.6721 - val_auc: 0.6810 - val_loss: 0.6449 - val_precision: 0.2250 - val_recall: 0.6609
Epoch 14/100
102/102 - 0s - 3ms/step - auc: 0.6895 - loss: 0.6365 - precision: 0.2384 - recall: 0.6721 - val_auc: 0.6789 - val_loss: 0.6439 - val_precision: 0.2176 - val_recall: 0.6264
Epoch 15/100
102/102 - 0s - 3ms/step - auc: 0.6860 - loss: 0.6375 - precision: 0.2400 - recall: 0.6597 - val_auc: 0.6760 - val_loss: 0.6436 - val_precision: 0.2267 - val_recall: 0.6437
Epoch 16/100
102/102 - 0s - 3ms/step - auc: 0.6863 - loss: 0.6388 - precision: 0.2407 - recall: 0.6492 - val_auc: 0.6775 - val_loss: 0.6458 - val_precision: 0.2303 - val_recall: 0.6207
Epoch 17/100
102/102 - 0s - 3ms/step - auc: 0.6909 - loss: 0.6353 - precision: 0.2444 - recall: 0.6635 - val_auc: 0.6784 - val_loss: 0.6437 - val_precision: 0.2247 - val_recall: 0.6379
Epoch 18/100
102/102 - 0s - 3ms/step - auc: 0.6950 - loss: 0.6336 - precision: 0.2463 - recall: 0.6730 - val_auc: 0.6815 - val_loss: 0.6407 - val_precision: 0.2380 - val_recall: 0.6552
Epoch 19/100
102/102 - 0s - 3ms/step - auc: 0.6951 - loss: 0.6321 - precision: 0.2455 - recall: 0.6606 - val_auc: 0.6802 - val_loss: 0.6493 - val_precision: 0.2265 - val_recall: 0.6379

```

```

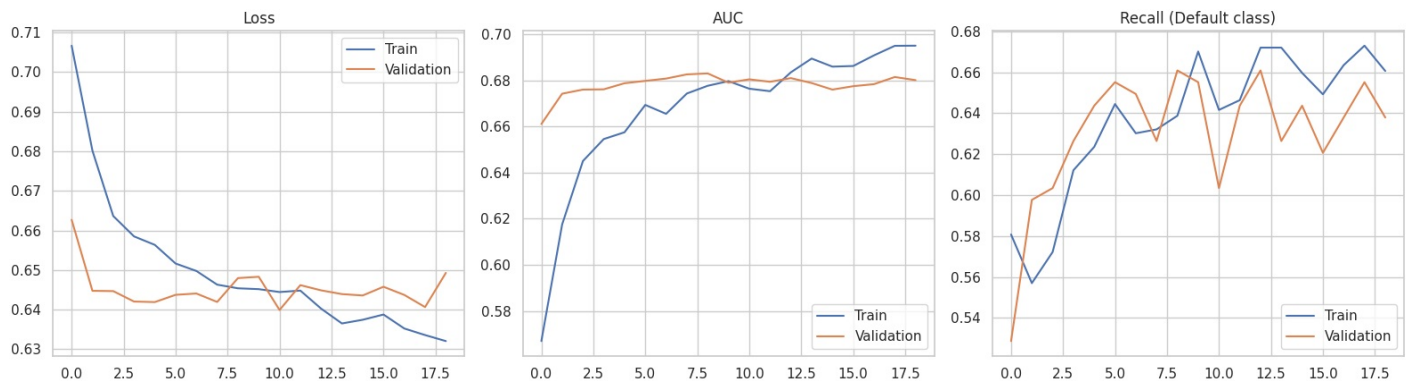
In [17]: fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
axes[0].plot(history.history['loss'], label='Train')
axes[0].plot(history.history['val_loss'], label='Validation')
axes[0].set_title('Loss')
axes[0].legend()

axes[1].plot(history.history['auc'], label='Train')
axes[1].plot(history.history['val_auc'], label='Validation')
axes[1].set_title('AUC')
axes[1].legend()

axes[2].plot(history.history['recall'], label='Train')
axes[2].plot(history.history['val_recall'], label='Validation')
axes[2].set_title('Recall (Default class)')
axes[2].legend()

plt.tight_layout()
plt.savefig('lc_training_curves.png', dpi=120)
plt.show()

```



Evaluation on the held-out test set

```

In [18]: y_prob = model.predict(X_test_scaled, verbose=0).ravel()
y_pred = (y_prob >= 0.5).astype(int)

test_auc = roc_auc_score(y_test, y_prob)
avg_precision = average_precision_score(y_test, y_prob)

print(f"Test AUC: {test_auc:.4f}")
print(f"Test Average Precision (PR-AUC): {avg_precision:.4f}")
print()
print(classification_report(y_test, y_pred, target_names=['Fully Paid', 'Not Fully Paid']))

```

Test AUC: 0.6814
 Test Average Precision (PR-AUC): 0.2922

	precision	recall	f1-score	support
Fully Paid	0.90	0.59	0.71	1609
Not Fully Paid	0.23	0.66	0.35	307
accuracy			0.60	1916
macro avg	0.57	0.62	0.53	1916
weighted avg	0.79	0.60	0.65	1916

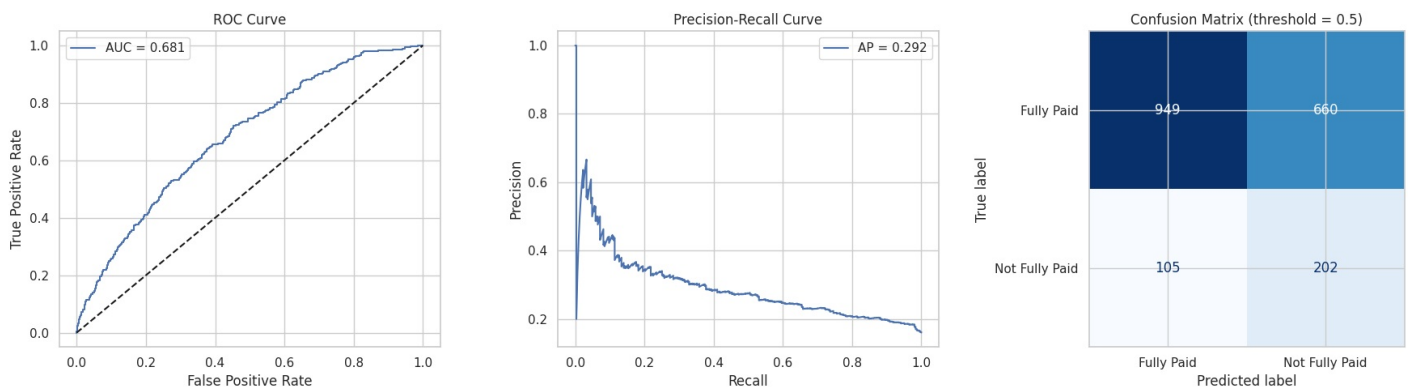
```
In [19]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))

fpr, tpr, _ = roc_curve(y_test, y_prob)
axes[0].plot(fpr, tpr, label=f'AUC = {test_auc:.3f}')
axes[0].plot([0,1],[0,1], 'k--')
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate')
axes[0].set_title('ROC Curve')
axes[0].legend()

prec, rec, _ = precision_recall_curve(y_test, y_prob)
axes[1].plot(rec, prec, label=f'AP = {avg_precision:.3f}')
axes[1].set_xlabel('Recall')
axes[1].set_ylabel('Precision')
axes[1].set_title('Precision-Recall Curve')
axes[1].legend()

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Fully Paid', 'Not Fully Paid'])
disp.plot(ax=axes[2], colorbar=False, cmap='Blues')
axes[2].set_title('Confusion Matrix (threshold = 0.5)')

plt.tight_layout()
plt.savefig('lc_evaluation_plots.png', dpi=120)
plt.show()
```



Interpreting the results: as in Section 2a, accuracy alone would be a misleading headline metric on this ~84:16 imbalanced target. The evaluation instead centers on **ROC-AUC** (ranking quality across all thresholds) and **Recall on the "Not Fully Paid" class** (the business-critical number — missing a true default costs LendingClub far more than an unnecessary manual review of a loan that would have been repaid). As with the earlier Home Credit project, a lender would typically want to move the decision threshold below 0.5 to trade some Precision for higher Recall on defaults, depending on the actual cost of a missed default versus a false alarm.

5. Summary

Key takeaways:

- Feature transformation:** the single categorical column (`purpose`) was one-hot encoded into discrete numerical features.
- EDA:** confirmed the ~84:16 class imbalance; found that lower `fico`, higher `int.rate`, and more recent credit inquiries are all associated with a higher chance of default — consistent with financial intuition and with LendingClub's own risk-based interest-rate pricing. `small_business` loans and loans that failed LendingClub's own credit-policy check default at noticeably higher rates.
- Correlation-based feature reduction:** checked pairwise feature correlations and removed redundant features

(keeping the one more predictive of the target), reducing dimensionality before modeling.

4. **Deep learning model:** a compact Keras/TensorFlow neural network ($64 \rightarrow 32 \rightarrow 16 \rightarrow 1$) with batch normalization, dropout, and class-weighted training was used to predict default risk, evaluated primarily via AUC and Recall on the minority (default) class rather than raw accuracy.

Possible next steps: threshold tuning to match LendingClub's actual cost structure for missed defaults vs. false alarms, and comparing this deep learning model against simpler baselines (e.g., logistic regression, gradient boosting) to confirm the added complexity of a neural network is justified on a dataset this size.